System architecture & logic

How a WhatsApp message becomes a calorie log — the full pipeline, the parser brain, the failure defenses, and everything that broke on the way here.

The one-sentence product: people fail at calorie tracking because opening an app after dinner is friction — so the tracker lives inside WhatsApp, where they already are. Users text what they ate in Hinglish; the bot replies with calories and macros in 2–5 seconds. No app, no account.

PART 1

The architecture at a glance

Six hops, all synchronous, one round trip. The entire response must fit inside Twilio's 15-second webhook window (it currently takes 2–5s).

User's phone

"2 roti aur dal" via WhatsApp

↓

Twilio WhatsApp Sandbox

Shared test number. Receives the message, fires an HTTP webhook.

↓

ngrok tunnel

Public URL → the Mac Mini at home. Free tier now gives a stable domain that survives restarts.

↓

Node/Express server — server.js

The brain. Rate limiting, dedupe, intent routing, reply formatting. Runs on the Mac Mini under launchd supervision.

↓ ↑

LLM chain — parser.js

Gemini → Groq → Claude. One call returns structured JSON: foods, quantities, and intent.

↓ ↑

Supabase (Postgres)

users · user_logs · foods_reference. Totals, meal clustering, corrections.

↓

TwiML reply

"✅ Logged: … Today: 458 kcal · P24g C66g F10g" — back through Twilio to the phone.

A key property: the reply is returned *inside* the webhook response (TwiML), so sending replies needs no API credentials and no separate send step. This is also why the bot kept "working" for a day while the Twilio credentials in `.env` were quietly wrong — nothing on the happy path ever used them.

PART 2

Life of a message

1. **Dedupe check.** Twilio retries webhooks that take >15s. Every message carries a unique `MessageSid`; repeats are dropped instantly so a retry can never double-log a meal.
2. **Gate checks, before any money is spent.** Message longer than 300 chars → bounced with a hint. Sender over 25 messages/hour or 60/day → polite rate-limit reply. Both fire before the LLM call, so abuse costs nothing.
3. **One LLM call.** The message (WhatsApp markdown stripped, emojis kept) goes to the parser with a ~3k-token system prompt. Back comes JSON: an `intent`, an items array with quantities and database matches, and parse notes.
4. **Intent routing.** `log` → insert rows. `replace_last` → delete the previous batch, insert the corrected food. `undo` → delete the last batch, show what was removed. A literal "undo" text skips the LLM entirely.
5. **Food resolution** (Part 4) turns each item into kcal + protein/carbs/fat.
6. **Totals & meal clustering.** Today's rows are fetched once (in parallel with the user upsert, saving a round trip); messages within 45 minutes of each other cluster into the same "meal" in the reply.
7. **Reply formatted and returned.** Every line shows what was assumed — "(est.)" markers, quantity multipliers, a macro legend.

PART 3

The parser brain

Everything the LLM knows arrives in one system prompt. Its design choices:

- **The food list is pipe-formatted, not JSON** — `ID 17 | Dal Tadka | aliases: dal, daal, tadka dal… | 180 kcal/bowl`. Around 60–70% fewer tokens than JSON, and the alias strings sit where the model's attention lands. Aliases are the single biggest lever on parse accuracy.
- **Intent classification is in the same call.** "sorry it was rajma", "make it 2x", "remove that last one" — the model labels every message `log` / `replace_last` / `undo`. Regex couldn't survive contact with real phrasing; the LLM catches wordings never seen before. Tie-break rule: when unsure, prefer `log` — a duplicate is recoverable, a wrongly deleted meal is not.
- **Quantity normalisation:** loose Hinglish maps to five discrete multipliers — *thodi si / adha* → 0.5, default 1.0, *sawa* → 1.5, *dabake / do* → 2.0, *teen / extra* → 3.0. Anything else is coerced to 1.0 server-side.
- **No-decomposition rule:** a named dish is ONE item. "Bhelpuri" must never become puri + sev + sabzi (that bug shipped 530 kcal for a 290 kcal snack on day one).
- **Quantity scope rule:** "2 roti with ghee" means 2 rotis and 1 ghee — a leading number binds only to the food it precedes.
- **Modifier rule:** ghee/butter/dahi split into separate items with their own lookup, so an unknown modifier never drags down the base food's match.
- **Annotation rule:** "bahot ghee tha biryani mai" is commentary on an *already logged* dish — log only the extra ghee, never re-log the biryani.
- **No-fabrication rule:** unknown food = `match_type "none"`, never an invented database ID. Prompt-injection attempts ("ignore all instructions…") fall out naturally: no food found → clarifying question, nothing logged.

PART 4

LLM fallback chain

Free-tier quotas die mid-day — proven twice in one day of testing (Groq's 100k tokens/day ≈ only ~33 messages at this prompt size; Gemini's per-model daily caps burned even faster). The chain is what keeps real users off the error path:

ORDER	PROVIDER	ROLE

1	Gemini 2.5 Flash	Primary. Free, resets daily.
2	Groq (Llama 3.3 70B)	Fallback. Free, fast (~1.3s).
3	Claude Haiku 4.5	Paid insurance. ~₹0.35/message from a $5 prepaid balance ≈ 1,200 messages.

Any failure — quota 429, server error, even malformed JSON — falls through to the next provider. A quota 429 fails over *instantly* (no pointless retry against a dead daily limit); transient 5xx gets one quick retry. If all three die, the user still gets a 300 kcal placeholder logged with an apology — never silence. Total worst-case fits the 15s window.

The economics: Claude-as-last-resort burns pennies per month while free tiers carry the load. The chain fired for real on day one and users never noticed.

PART 5

Food → calories: four tiers, never a dead end

> **Tier 1 — Curated list (67 foods)**
>
> Hand-checked Indian staples with Hinglish aliases, living inside the system prompt. Instant, highest-quality data. Grows demand-driven: every unmatched food in the logs is a candidate.

> **Tier 2 — INDB reference (1,014 recipes)**
>
> The full Indian Nutrient Databank (open-access, lab-derived) imported into Supabase. Unknown foods get fuzzy-matched via Postgres trigram search (`match_food` RPC, strict 0.75 threshold — a false match with confident macros is worse than a miss).

> **Tier 3 — LLM estimate**
>
> The model's own per-serving guess (papad ≈50, shawarma ≈450), clamped to 20–800 kcal so a hallucination can't poison a day's total.

> **Tier 4 — 300 kcal floor**
>
> Zero information → "meal — 300 kcal (est.)". The bot must always log *something*; it never asks the user to guess calories.

Dirty-data guard: some INDB "per serving" values are whole-recipe yields (its Poori row claims 921 kcal per poori). Serving values are trusted only in the 20–800 range;

otherwise the per-100g value ×1.5 is used. Dishes whose INDB rows are junk in both columns (kadhi: 64g fat/100g) get promoted to the curated list with hand-checked numbers instead.

Accuracy ceiling, communicated honestly: home-cooked Indian food is ±15–20% on any tracker, because databases hold averages, not your kitchen. The product targets directional awareness — consistent logging drives behavior change, not clinical precision.

PART 6

Data model

TABLE	HOLDS
`users`	Phone number (the only identity), goal, calibration sizes, created_at. No signup — first message creates the row.
`user_logs`	Every logged item: food, quantity, kcal, P/C/F, matched_db_id, is_estimate, timestamp, IST date. Rows with `matched_db_id = null` double as the queue of foods to add next.
`foods_reference`	The 1,014 INDB recipes with per-serving and per-100g nutrition + trigram index.
`food_preferences`	Reserved for one-time calibration answers ("chota katori or bada bowl?") — asked once per food type, ever.

Corrections are pure row operations: `replace_last` deletes the last batch (grouped by identical timestamp) and inserts the new parse. A name-only correction ("it was veg not chicken") inherits the old quantity — the 2x survives the swap. No conversation state is stored anywhere; every message is self-contained, which is what makes the whole system this simple.

PART 7

Defenses

- **Rate caps:** 25 messages/hour, 60/day per phone — checked before the LLM spend. One spammer can't burn the shared quota.
- **Length cap:** 300 chars. Long pastes bounce for free.
- **Webhook dedupe** by MessageSid (in-memory, last 1,000).

- **Injection handling:** zero-item parses get a clarifying question, never a log — unless parse notes show a genuine system failure, in which case the placeholder fires (the user probably did send a meal).
- **Undo** as the universal escape hatch: fat-finger recovery in one word.

PART 8

Operations: the Mac Mini setup

Three launchd agents (macOS's native supervisor) own everything. Versioned copies live in the repo under `launchd/`.

AGENT	DOES
`…server`	`caffeinate -dims node server.js` — KeepAlive restarts it within seconds of any crash. Kill-tested: back in 6s.
`…ngrok`	The tunnel, same KeepAlive. The free stable domain means restarts no longer change the URL.
`…healthcheck`	Every 5 min: pings the public URL and checks Twilio balance. Failure → WhatsApp alert to Swapnil via Twilio's API (works precisely when the tunnel is what's broken). Max one alert/hour per problem.

- **Logs** live in `~/Library/Logs/` (nutridesi.log, -ngrok.log, -healthcheck.log) — *not* the project folder, because launchd opens log files as itself and can't touch ~/Documents (fails with status 78; learned the hard way).
- **Watch live traffic:** `tail -f ~/Library/Logs/nutridesi.log | grep ms`
- **Manual restart:** `launchctl kickstart -k gui/501/com.nutridesi.server` — never start node by hand; launchd owns the port.
- **Power:** `pmset -c sleep 0` + caffeinate belt-and-braces; auto-login + `autorestart 1` means even a power cut ends with the bot back online untouched.

PART 9

Battle scars — what actually broke, and the fix

Every defense above exists because something failed first. The honest log:

> **Bhelpuri decomposed into 3 ingredients (530 vs 290 kcal)**

First real user, first hour. Fix: the no-decomposition rule + Bhel Puri added to the curated list.

Corrections stacked instead of replacing

"Sorry it was pani puri" added a 6th entry on top of 5 wrong ones. Fix: LLM intent classification with delete-then-insert semantics.

Two LLM free tiers died in one afternoon

Groq's daily tokens ≈33 messages; Gemini's per-model cap even less. Fix: the three-provider chain.

Papad logged as 300 kcal (reality: ~50)

The flat placeholder inflated a daily habit by 250 kcal/day — the exact error the product exists to prevent. Fix: LLM estimates + INDB tier.

"papad" fuzzy-matched "Raw papaya with coconut"

Trigram threshold too loose. Fix: 0.75 cutoff — no match beats a confident wrong match.

Replies silently stopped mid-launch-day

Twilio trial cap: 9 outbound messages/day, undocumented in the console. The bot logged everything, replied to nothing. Fix: account upgrade — then a compliance suspension (error 90010), resolved with a public GitHub repo as verification. Two days of launch drama, zero code at fault.

The Mac Mini slept after 1 minute

Headless Mini idle-slept and took the bot with it. Fix: pmset + caffeinate.

The meta-lesson: every failure was found by *us* before a stranger hit it — quota exhaustion during our own testing, the Twilio cap the day before the group launch. Pre-launch chaos was the cheapest possible place to buy these lessons.

PART 10

Known limits & what's next

- **72-hour sandbox re-join:** users who go quiet for 3 days must resend the join code. Unfixable inside the sandbox — the real fix is a proper WhatsApp Business number

(Meta Cloud API or Twilio WABA), which is the graduation step if retention shows up.

- **The Mac Mini is a single point of failure:** the watchdog can't alert from a dead machine. External uptime monitor (free) is the cheap patch; Railway deployment is the real one.
- **No query answering yet:** "what's my total?" gets "what did you eat?" — testers already ask for it; likely first Week-2 feature.
- **The metric that decides everything:** D7 retention ≥ 40% validates the WhatsApp-friction hypothesis; < 20% kills it. Parser direct-match ≥ 80% or the alias map needs work before more users. One person logging food two days in a row beats twenty "cool bro" replies.

`BUILT IN A WEEKEND · HARDENED IN TWO DAYS OF FIRE · github.com/swamagic2525/nutridesi`